Formalization Architecture and Lean Infrastructure for the Finite-Alphabet Rate–Distortion Theorem

Fubin Yan

Verified reference, 14 August 2026

Abstract

This document describes the Lean 4 formalization of the finite-alphabet rate–distortion theorem in `Myproject/RateDistortion/RateDistortion.lean`. The development is organized as reusable information-theoretic infrastructure, not as a single proof script for the final theorem. It builds finite-PMF expectation and variance, Shannon and conditional entropy, a bridge from mathlib's measure-theoretic Kullback–Leibler divergence to finite entropy formulas, memoryless block sources and channels, information density, weak-law bounds, random-codebook covering, deterministic codebook extraction, single-letterization for arbitrary deterministic codes, asymptotic code sequences, and a diagonal argument that removes the positive coding slacks.

The final public API proves strict achievability whenever $R_I(P, d, D) < R$ and D is feasible, proves the converse $R_I(P, d, D) \le R$ on the interior distortion region, and derives strict nonachievability below the information rate–distortion function. The report states the mathematical and Lean versions of the key definitions and theorems, explains the principal mathlib reuse decisions, records the exact scope of the formalization, and separates the proved strict statements from the still-open boundary-rate question $R = R_I(P, d, D)$.

Contents

1 Purpose and contribution **1**

2 Architecture at a glance **2**

3 Formal setting and representation choices **4**
3.1 Finite alphabets 4
3.2 Distortion and boundedness 4
3.3 Blocks and positive blocklengths 5

4 Finite-PMF probability infrastructure **5**

5 Entropy, KL divergence, and mutual information **6**
5.1 Shannon and conditional entropy 6
5.2 KL-based and entropy-based mutual information 6

6 The single-letter rate–distortion function **7**

7 Memoryless block infrastructure **8**

8 Information density and the direct coding theorem **9**
8.1 Information density 9
8.2 Good pairs and random covering 9
8.3 Deterministic extraction and the finite-block direct theorem 10

9 Arbitrary codes and the finite-block converse **10**

10 Operational code sequences and diagonalization **11**
10.1 Positive-blocklength code sequences 11
10.2 Why a diagonal construction is required 11

11 Main theorem API and exact scope **12**
11.1 Strict achievability 12
11.2 Converse with positive distortion slack 12
11.3 Interior converse and strict nonachievability 13

12 Reusable infrastructure **13**

13 Mathlib reuse and type discipline **14**

14 Verification and reproducibility **15**

15 Current scope and extension path **15**

16 Conclusion **16**

1 Purpose and contribution

The classical rate–distortion theorem relates two descriptions of lossy source coding. The information quantity is

$$R_I(P, d, D) = \inf_{W:\, \mathbb{E}_{PW}[d(X,Y)] \le D} I_{PW}(X; Y),$$

where P is a source distribution and $W(y \mid x)$ is a test channel. The operational quantity is obtained from sequences of finite-block deterministic encoders and decoders whose asymptotic rate and expected distortion are bounded.

The formalization has two goals:

1. verify the direct and converse proof chains for finite alphabets;
2. provide stable finite-information-theory APIs that later developments can reuse for channel coding, source coding, typicality, concentration, and other single-letterization arguments.

Lean forces several conventions to be explicit. Blocklength zero cannot be silently divided away; distortion values and possibly infinite thresholds must live in compatible ordered types; the KL and entropy definitions must be shown to agree; the infimum defining R_I need not be attained; a fixed-slack code for every sufficiently large blocklength is not yet a single sequence with vanishing slack; and an asymptotic converse at $D + \varepsilon$ does not imply a converse at D without a continuity argument. These issues determine the architecture of the Lean code.

2 Architecture at a glance

Figure 1 is a declaration-level dependency graph extracted from the compiled Lean environment. It is not a manually asserted outline: every arrow is backed by a constant reference in a declaration type, definition body, or theorem proof. Hidden helper declarations are traversed until another selected public declaration is reached, and the resulting graph is transitively reduced. Thus an arrow $A \to C$ is omitted when the displayed graph already contains a path $A \to \cdots \to C$.

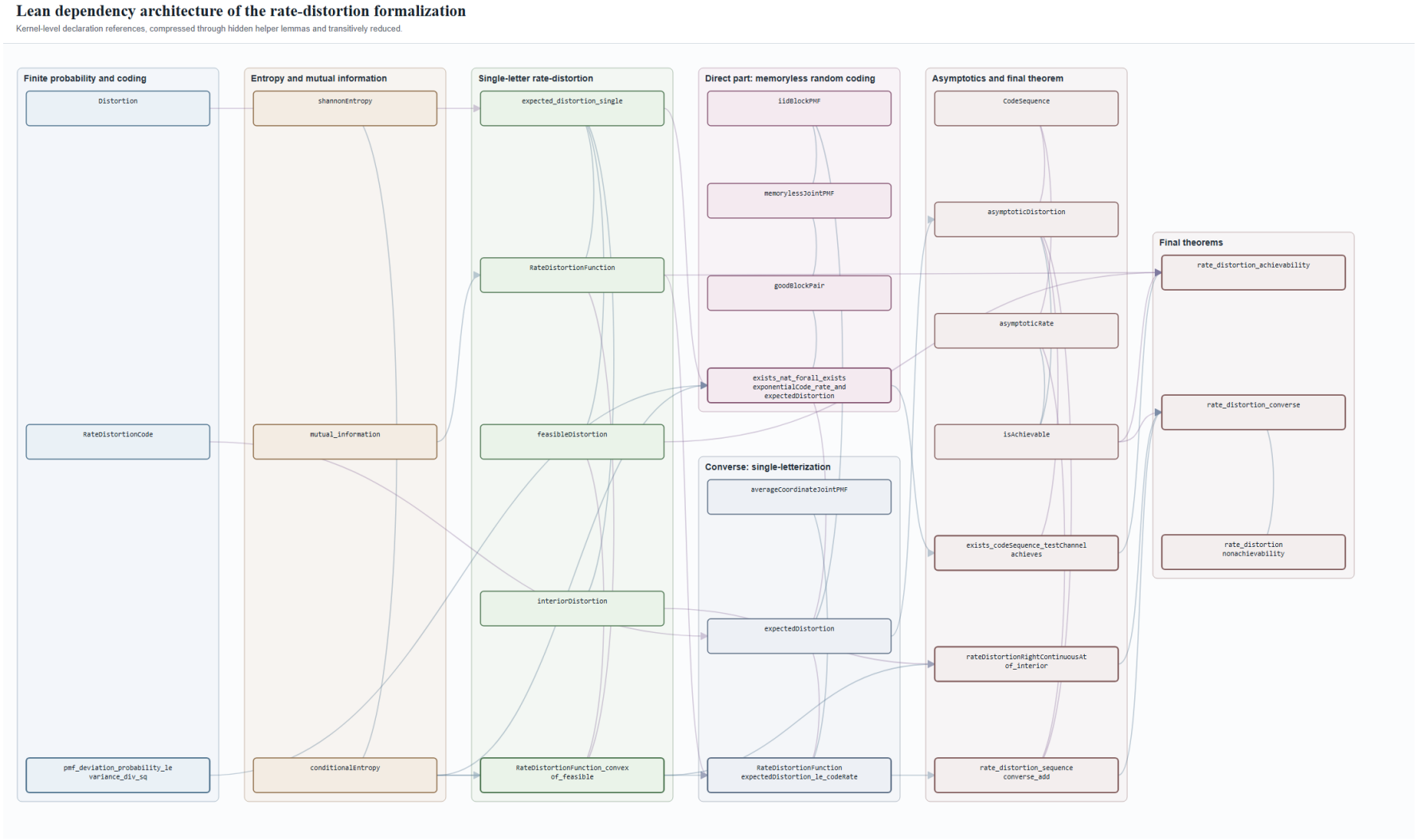


Figure 1: Machine-extracted Lean dependency architecture. Arrows point from prerequisites to declarations that use them. Solid edges arise from proof terms or definition bodies, dashed edges from declaration types, and purple edges from both. The report view displays 28 core declarations and 42 reduced edges; the accompanying interactive graph exposes all 38 selected declarations.

The reproducible extraction script is `GenerateDependencyGraph.lean`. It uses Lean's `ConstantInfo.getUsedConstantsAsSet` API on the compiled module. The machine-readable output is `rate_distortion_dependencies.json`; `GenerateDependencyArtifacts.ps1` merges duplicate statement/proof edges and computes the transitive reduction. The stand-alone file `rate_distortion_dependency_graph.html` provides search, core/full views, edge-type filtering, and clickable upstream/downstream inspection without an external web service.

There are two distinct proof-oriented endpoints. The direct part constructs codes from a test channel and then diagonalizes shrinking error tolerances. The converse part turns an arbitrary block code into a single-letter joint law and then passes its finite-block inequality to a limsup. They meet only at the single-letter function `RateDistortionFunction` and at the operational definition `isAchievable`.

The current implementation is a single source file. Its internal layering is mathematically coherent, but the dependency graph suggests a future split into modules for finite PMFs, entropy, memoryless blocks, random coding, converse, and asymptotics. Section 15 records that extension path.

3 Formal setting and representation choices

3.1 Finite alphabets

The entire development works with finite source and reproduction alphabets:

```
variable {X Y : Type*}
  [Fintype X] [Fintype Y] [DecidableEq X] [DecidableEq Y]
```

The use of `Fintype` makes probabilities, expectations, entropy, and event probabilities finite sums. This permits an elementary random-coding formalization while still using mathlib's canonical `PMF` type. Theorems such as `nonempty_of_pmf` and `nonempty_reproduction_of_source_channel` recover nonemptiness from actual probability data instead of adding global `Nonempty` assumptions.

3.2 Distortion and boundedness

The single-letter distortion is finite and nonnegative:

$$d : X \times Y \longrightarrow \mathbb{R}_{\geq 0}.$$

```
def Distortion := X -> Y -> NNReal

noncomputable def distortionBound
    (d : Distortion (X := X) (Y := Y)) : NNReal :=
  Finset.univ.sup (fun x => Finset.univ.sup (fun y => d x y))

theorem distortion_le_bound
    (d : Distortion (X := X) (Y := Y)) (x : X) (y : Y) :
    d x y <= distortionBound d
```

Choosing `NNReal` rather than `ENNReal` for each value of d rules out an infinite letterwise distortion. Because both alphabets are finite, the global bound $d_{\max}$ is then derived rather than assumed. Expected distortions and distortion thresholds use `ENNReal`; hence limits, infima, and the value $\top$ remain available at the analytic layer.

This division of roles is important:

$$d(x, y) \in \mathbb{R}_{\geq 0}, \qquad \mathbb{E}[d] \in [0, \infty], \qquad D \in [0, \infty].$$

The code proves that all expectations generated by finite distortion remain below the finite coercion of $d_{\max}$.

3.3 Blocks and positive blocklengths

Blocks are functions on `Fin n`:

```
abbrev Xblock (n : Nat) (X : Type*) := Fin n -> X
abbrev Yblock (n : Nat) (Y : Type*) := Fin n -> Y
abbrev Index (M : Nat) := Fin M
```

For $n > 0$, block distortion and code rate are

$$d_n(x^n, y^n) = \frac{1}{n} \sum_{i \in \mathrm{Fin}(n)} d(x_i, y_i), \qquad r(n, M) = \frac{\log M}{n}.$$

```
def blockDistortion (hn : 0 < n)
    (d : Distortion (X := X) (Y := Y))
    (x : Xblock n X) (y : Yblock n Y) : ENNReal :=
  (∑ i : Fin n, (d (x i) (y i) : ENNReal)) / n

def codeRate (hn : 0 < n) : Real := Real.log M / n
```

The proof witness `hn : 0 < n` is part of both definitions. At the sequence level the actual blocklength at index k is $k + 1$, so zero-length codes never enter the operational theorem.

4 Finite-PMF probability infrastructure

The formalization introduces a small real-valued expectation and variance API for finite PMFs:

$$\mathbb{E}_P[f] = \sum_a P(a) f(a), \qquad \mathrm{Var}_P(f) = \sum_a P(a)(f(a) - \mathbb{E}_P[f])^2.$$

```
noncomputable def pmfRealExpectation
    (P : PMF A) (f : A -> Real) : Real :=
  ∑ a : A, (P a).toReal * f a

noncomputable def pmfRealVariance
    (P : PMF A) (f : A -> Real) : Real :=
  ∑ a : A, (P a).toReal *
    (f a - pmfRealExpectation P f) ^ 2
```

The corresponding reusable API includes:

- normalization in `ENNReal` and after `toReal`;
- expectation under `PMF.map`;
- linearity for addition and scalar multiplication;
- invariance of variance under a PMF map;
- product-distribution expectation and variance formulas;
- a finite-sum Chebyshev inequality,

$$P\{|f - \mathbb{E}f| \geq \varepsilon\} \leq \frac{\mathrm{Var}(f)}{\varepsilon^2}.$$

The Chebyshev theorem is stated directly as a finite PMF sum. This is a deliberate boundary: no measurable-space structure is needed for elementary finite concentration, while the KL layer below still reuses mathlib's measure-theoretic information theory.

Other broadly reusable finite-PMF lemmas include change of variables for `map`, a law-of-total-expectation identity for `bind`, support transport, and the probabilistic-method lemma `exists_le_pmf_weighted_sum`, which extracts an outcome whose value is no greater than its PMF-weighted average.

5 Entropy, KL divergence, and mutual information

5.1 Shannon and conditional entropy

Entropy is real-valued and measured in nats:

$$H(P) = -\sum_x P(x) \log P(x).$$

```
noncomputable def shannonEntropy (P : PMF X) : Real :=
  ∑ x : X, Real.negMulLog (P x).toReal

noncomputable def conditionalEntropy (Q : PMF (X × Y)) : Real :=
  shannonEntropy Q - shannonEntropy (Q.map Prod.fst)
```

The entropy infrastructure proves nonnegativity, the cardinality bound $H(P) \le \log |X|$, entropy decrease under deterministic maps, invariance under injective maps, the product formula, joint subadditivity, strong subadditivity, and "conditioning reduces entropy." The block converse later reuses the coordinate form

$$H(Y^n \mid X^n) \le \sum_{i=1}^{n} H(Y_i \mid X_i).$$

5.2 KL-based and entropy-based mutual information

The primary extended-nonnegative mutual information is built from mathlib's KL divergence:

$$I_Q(X;Y) = D_{\mathrm{KL}}(Q \,\|\, Q_X \otimes Q_Y).$$

```
noncomputable def kl_divergence (P Q : PMF X) : ENNReal :=
  letI : MeasurableSpace X := top
  InformationTheory.klDiv P.toMeasure Q.toMeasure

def product_pmf (P_X : PMF X) (P_Y : PMF Y) : PMF (X × Y) :=
  P_X.bind (fun x => P_Y.map (fun y => (x, y)))

noncomputable def mutual_information (Q : PMF (X × Y)) : ENNReal :=
  kl_divergence Q (product_pmf (Q.map Prod.fst) (Q.map Prod.snd))
```

For algebraic finite proofs, the development also uses

$$I_H(Q) = H(Q_X) + H(Q_Y) - H(Q).$$

```
noncomputable def mutualInformationEntropy (Q : PMF (X × Y)) : Real :=
  shannonEntropy (Q.map Prod.fst) +
```

```
    shannonEntropy (Q.map Prod.snd) - shannonEntropy Q

theorem mutual_information_eq_ofReal_entropy (Q : PMF (X × Y)) :
  mutual_information Q =
    ENNReal.ofReal (mutualInformationEntropy Q)
```

This bridge is a central interface theorem. Its proof establishes absolute continuity of a finite joint PMF with respect to the product of its marginals, identifies the Radon–Nikodym derivative with an explicit PMF density ratio, and evaluates mathlib's log-likelihood integral as a finite entropy sum. Consequently downstream proofs can use real arithmetic for entropy identities and return to `ENNReal` at public boundaries.

The same layer defines conditional PMFs from arbitrary joint laws and proves that every finite joint PMF is reconstructed as the joint induced by its first marginal and conditional channel. This is later used to formulate the rate–distortion function both through channels and through arbitrary joint laws with prescribed source marginal.

6 The single-letter rate–distortion function

For a source PMF P, distortion d, and threshold D, the information rate–distortion function is the infimum over test channels:

$$R_I(P, d, D) = \inf\{I(P; W) : \mathbb{E}_{PW}[d(X, Y)] \le D\}.$$

```
noncomputable def RateDistortionFunction
    (P : PMF X) (d : Distortion (X := X) (Y := Y))
    (D : ENNReal) : ENNReal :=
  sInf {r : ENNReal | exists W : X -> PMF Y, And
    (expected_distortion_single P W d <= D)
    (mutual_information (induced_joint P W) = r)}
```

The definition uses `sInf` rather than assuming an optimizer. This choice supports the exact selection theorem needed by strict achievability:

```
theorem exists_testChannel_of_RateDistortionFunction_lt_ofReal
    (hR : RateDistortionFunction P d D < ENNReal.ofReal R) :
  exists W : X -> PMF Y, And
    (expected_distortion_single P W d <= D)
    (mutualInformationEntropy (induced_joint P W) < R)
```

The main analytic API proves:

- `RateDistortionFunction_le`: every admissible test channel supplies an upper bound;
- `RateDistortionFunction_antitone`: $D_1 \le D_2$ implies $R_I(D_2) \le R_I(D_1)$;
- `RateDistortionFunction_convex_of_feasible`: channel mixtures give convexity on feasible distortion levels;
- finiteness of $R_I(D)$ at feasible levels;
- $R_I(D) = 0$ strictly above the zero-rate distortion threshold;
- right continuity on the interior distortion region.

The distortion-domain predicates are:

```
def feasibleDistortion (P : PMF X) (d : Distortion) (D : ENNReal) : Prop :=
  exists W : X -> PMF Y, expected_distortion_single P W d <= D
```

```
noncomputable def minimumDistortion (P : PMF X) (d : Distortion) : ENNReal :=
  sInf (Set.range (fun W : X -> PMF Y =>
    expected_distortion_single P W d))

noncomputable def zeroRateDistortion (P : PMF X) (d : Distortion) : ENNReal :=
  sInf (Set.range (fun Q : PMF Y =>
    expected_distortion_single P (fun _ => Q) d))

def interiorDistortion (P : PMF X) (d : Distortion) (D : ENNReal) : Prop :=
  And (minimumDistortion P d < D) (D < zeroRateDistortion P d)
```

Right continuity is proved rather than postulated. Given a feasible point $D_0 < D$, the proof mixes a channel at D_0 with a channel at $D + \varepsilon$. The barycentric weight $\varepsilon/(D - D_0 + \varepsilon)$ tends to zero, and convexity squeezes $R_I(D + \varepsilon)$ to $R_I(D)$ as $\varepsilon \downarrow 0$.

7 Memoryless block infrastructure

The i.i.d. source and memoryless test channel are explicit finite PMFs:

$$P^n(x^n) = \prod_i P(x_i), \qquad W^n(y^n \mid x^n) = \prod_i W(y_i \mid x_i).$$

```
noncomputable def iidBlockPMF (P : PMF X) : PMF (Xblock n X)

noncomputable def memorylessChannelBlockPMF
    (W : X -> PMF Y) (x : Xblock n X) : PMF (Yblock n Y)

def memorylessJointPMF (P : PMF X) (W : X -> PMF Y) :
    PMF (Xblock n X × Yblock n Y) :=
  induced_joint (iidBlockPMF n P) (memorylessChannelBlockPMF n W)
```

The block layer proves coordinate marginals, output marginals, support transport, preservation of single-letter expected distortion, entropy additivity, and

$$I(X^n; Y^n) = nI(X; Y).$$

The equivalence

$$(X^n \times Y^n) \simeq (X \times Y)^n$$

is represented by `blockPairEquiv`. It allows expectation and variance results for i.i.d. scalar samples to be reused for paired source–reproduction blocks instead of reproving product-sum identities in a second coordinate representation.

The generic block-statistics API proves

$$\mathbb{E}\left[\sum_{i=1}^{n} f(Z_i)\right] = n\mathbb{E}[f(Z)], \qquad \mathrm{Var}\left(\frac{1}{n}\sum_{i=1}^{n} f(Z_i)\right) = \frac{\mathrm{Var}(f(Z))}{n},$$

and derives the corresponding weak-law estimate by Chebyshev. This machinery is reused twice in the direct theorem: once for distortion and once for information density.

8 Information density and the direct coding theorem

8.1 Information density

The single-letter information density is the log density ratio

$$\imath_{P,W}(x;y) = \log \frac{P(x)W(y \mid x)}{P(x)(PW)(y)}.$$

```
noncomputable def singleLetterInformationDensity
    (P : PMF X) (W : X -> PMF Y) (z : X × Y) : Real :=
  Real.log (pmfDensityRatio (induced_joint P W)
    (product_pmf P (P.bind W)) z).toReal
```

Its expectation is $I(P;W)$. On the support of the memoryless joint law, the block information density factors as

$$\imath_{P^n,W^n}(x^n;y^n) = \sum_{i=1}^{n} \imath_{P,W}(x_i;y_i).$$

This equality is paired with the variance API to obtain a weak-law bound for $n^{-1}\imath(X^n;Y^n)$.

8.2 Good pairs and random covering

For a distortion threshold D' and information threshold t, a block pair is good when

$$d_n(x^n,y^n) \le D', \qquad \imath_{P^n,W^n}(x^n;y^n) \le t.$$

```
noncomputable def goodBlockPair
    (hn : 0 < n) (P : PMF X) (W : X -> PMF Y)
    (d : Distortion) (distortionThreshold : ENNReal)
    (informationThreshold : Real)
    (x : Xblock n X) (y : Yblock n Y) : Prop :=
  And (blockDistortion n hn d x y <= distortionThreshold)
    (memorylessInformationDensity n P W (x, y) <= informationThreshold)
```

Chebyshev's inequality makes the joint good-pair probability tend to one under fixed positive distortion and information slacks. A finite averaging lemma then shows that all but source mass 2δ has conditional good-set probability at least $1/2$ under the test channel.

The codebook distribution consists of M independent samples from the output marginal $(PW)^n$:

```
noncomputable def randomCodebookPMF
    (M n : Nat) (Q : PMF (Yblock n Y)) :
    PMF (Index M -> Yblock n Y) :=
  iidBlockPMF M Q
```

The information-density bound changes measure from $W^n(\cdot \mid x^n)$ to $(PW)^n$. If the test channel assigns at least $1/2$ to the good set and the information threshold is nt, then one random codeword hits that set with probability at least $\frac{1}{2}e^{-nt}$. Therefore an i.i.d. codebook misses it with probability at most

$$\left(1 - \frac{1}{2}e^{-nt}\right)^M \le \exp\left(-\frac{M}{2}e^{-nt}\right).$$

The chosen codebook size is

$$M_n = \lfloor e^{nR} \rfloor,$$

implemented by the Lean definition

`exponentialCodebookSize`.

Rounding down gives the pointwise rate bound $\log M_n/n \le R$, while for large n and $t < R$ it still gives the covering estimate

$$P_{\text{miss}} \le \exp\left(-\frac{1}{4}e^{n(R-t)}\right).$$

8.3 Deterministic extraction and the finite-block direct theorem

The encoder selects the first good codeword and falls back to index zero when none exists. The global distortion bound controls the fallback loss. Averaging over random codebooks and applying the probabilistic method extracts a deterministic codebook with no larger miss probability.

The principal finite-block theorem is `exists_nat_forall_exists_exponentialCode_rate_and_expectedDistortion`. In mathematical form, if

$$\mathbb{E}_{PW} d + \varepsilon_d \le D', \qquad I(P;W) + \varepsilon_i \le t < R,$$

then for all sufficiently large n there is a deterministic code of size $\lfloor e^{nR} \rfloor$ satisfying

$$r_n \le R, \qquad \mathbb{E}[d_n] \le D' + d_{\max}\left(2\delta + \exp\left[-\frac{1}{4}e^{n(R-t)}\right]\right).$$

This theorem is deliberately stronger than a bare asymptotic existence statement. It exposes an explicit error bound and supplies a code at every sufficiently large blocklength, which makes it reusable independently of the final diagonal argument.

9 Arbitrary codes and the finite-block converse

A deterministic block code is represented without extra stochastic structure:

```
structure RateDistortionCode
    (X Y : Type*) [Fintype X] [Fintype Y] (n M : Nat) where
  encoder : Xblock n X -> Index M
  decoder : Index M -> Yblock n Y
```

For a source block $X^n \sim P^n$, the reproduction is $\widehat{Y}^n = g_n(f_n(X^n))$. The development defines the joint block law, each coordinate joint law $Q_i = \mathcal{L}(X_i, \widehat{Y}_i)$, and the uniform time-sharing mixture

$$\overline{Q} = \frac{1}{n}\sum_{i=1}^{n} Q_i.$$

The converse infrastructure proves:

1. the source marginal of every Q_i and of $\overline{Q}$ is P;
2. the expected distortion of $\overline{Q}$ equals the expected block distortion of the code;
3. $H(\widehat{Y}^n) \le H(f_n(X^n)) \le \log M$;
4. strong subadditivity and the i.i.d. source law imply $\sum_i I(Q_i) \le I(X^n; \widehat{Y}^n)$;
5. convexity under time sharing gives $I(\overline{Q}) \le n^{-1}\sum_i I(Q_i)$.

The resulting finite-block interface is:

```
theorem RateDistortionFunction_expectedDistortion_le_codeRate
    (hn : 0 < n) (code : RateDistortionCode X Y n M)
    (P : PMF X) (d : Distortion) :
  RateDistortionFunction P d
      (expectedDistortion n M hn code P d) <=
    ENNReal.ofReal (codeRate n M hn)
```

Mathematically,

$$R_I\Big(P, d, \mathbb{E}[d_n(X^n, \widehat{Y}^n)]\Big) \le \frac{\log M}{n}.$$

This theorem is reusable for any deterministic finite-block code; it does not depend on the random-coding construction used in the direct part.

10 Operational code sequences and diagonalization

10.1 Positive-blocklength code sequences

```
structure CodeSequence (X Y : Type*) [Fintype X] [Fintype Y] where
  M : Nat -> Nat
  hM : forall n, 0 < M n
  code : forall n, RateDistortionCode X Y (n + 1) (M n)
```

The sequence index n represents the actual blocklength $n+1$. This avoids a special zero-length branch in every asymptotic definition while retaining one code for each positive blocklength.

The asymptotic quantities are mathlib filter limsups:

$$\overline{r}(\mathcal{C}) = \limsup_{n\to\infty} \mathrm{ofReal}(r_n), \qquad \overline{d}(\mathcal{C}) = \limsup_{n\to\infty} \mathbb{E}[d_n].$$

```
def asymptoticDistortion (seq : CodeSequence X Y)
    (P : PMF X) (d : Distortion) : ENNReal :=
  Filter.limsup (fun n => expectedDistortion (n + 1) (seq.M n)
    (Nat.succ_pos n) (seq.code n) P d) atTop

def asymptoticRate (seq : CodeSequence X Y) : ENNReal :=
  Filter.limsup (fun n => ENNReal.ofReal
    (codeRate (n + 1) (seq.M n) (Nat.succ_pos n))) atTop

def isAchievable (P : PMF X) (d : Distortion)
    (R : Real) (D : ENNReal) : Prop :=
  exists seq : CodeSequence X Y, And
    (asymptoticRate seq <= ENNReal.ofReal R)
    (asymptoticDistortion seq P d <= D)
```

The construction internally proves the stronger pointwise rate condition `hasPointwiseRateBound`; a separate theorem promotes it to the limsup bound in `isAchievable`.

10.2 Why a diagonal construction is required

A fixed positive slack only produces

$$\limsup_n \mathbb{E}[d_n] \le D + \eta + 3d_{\max}\delta.$$

One cannot choose $\eta = \delta = 0$ in the finite-block concentration and covering lemmas. Moreover, choosing a different eventual sequence for every (η, δ) does not by itself produce one sequence attaining D.

The formalization uses the standard schedule

$$\eta_k = \delta_k = \frac{1}{k+1}$$

and chooses a threshold N_k after which the level-k code exists. At blocklength $n+1$, the selector

`activePrecisionLevel`

chooses the largest precision level whose threshold has been reached. Every fixed level is eventually active, so the selected level tends to infinity. The error

$$e_k = \mathrm{ofReal}(\eta_k) + d_{\max}\, 3\, \mathrm{ofReal}(\delta_k)$$

tends to zero.

The abstract reusable pair of lemmas is:

- `exists_diagonalCodeSequence_of_eventually_codes`, which turns level-dependent eventual code families into one full sequence;
- `asymptoticDistortion_le_of_diagonal_error`, which converts an eventual bound $D + e_{k(n)}$ and $e_k \to 0$ into a limsup bound D.

These lemmas do not mention mutual information or random codebooks. They form a general diagonalization API for asymptotic coding theorems with shrinking precision parameters.

11 Main theorem API and exact scope

11.1 Strict achievability

```
theorem rate_distortion_achievability
    (P : PMF X) (d : Distortion) (D : ENNReal) (R : Real)
    (hD : feasibleDistortion P d D)
    (hR : RateDistortionFunction P d D < ENNReal.ofReal R) :
    isAchievable P d R D
```

Thus every feasible pair strictly above the information rate–distortion function is operationally achievable:

$$R_I(P, d, D) < R \quad \Longrightarrow \quad (R, D) \text{ is achievable.}$$

For finite D, strictness selects a test channel with $I(P; W) < R$, after which the random-coding and diagonal APIs construct the required sequence. The $D = \top$ branch is handled separately by a constant reconstruction sequence; feasibility provides a nonempty reproduction alphabet.

11.2 Converse with positive distortion slack

For every achievable sequence and every $\varepsilon > 0$, the formalization first proves the unconditional asymptotic statement

$$R_I(P, d, D + \varepsilon) \le R.$$

```
theorem rate_distortion_converse_add
    (hEps : 0 < epsilon) (hA : isAchievable P d R D) :
  RateDistortionFunction P d (D + epsilon) <= ENNReal.ofReal R
```

This is the direct consequence of the finite-block converse and limsup calculus; it does not require a continuity hypothesis.

11.3 Interior converse and strict nonachievability

Right continuity is established on

$$D_{\min} < D < D_0,$$

where $D_{\min}$ is `minimumDistortion` and D_0 is `zeroRateDistortion`. Therefore:

```
theorem rate_distortion_converse
    (hDint : interiorDistortion P d D)
    (hA : isAchievable P d R D) :
  RateDistortionFunction P d D <= ENNReal.ofReal R

theorem rate_distortion_nonachievability
    (hDint : interiorDistortion P d D)
    (hR : ENNReal.ofReal R < RateDistortionFunction P d D) :
  not (isAchievable P d R D)
```

The two final directions are therefore:

$$\begin{aligned} R > R_I(P,d,D) \text{ and } D \text{ feasible} &\implies (R,D) \text{ achievable}, \\ R < R_I(P,d,D) \text{ and } D \text{ interior} &\implies (R,D) \text{ not achievable}. \end{aligned}$$

The formalization intentionally does *not* claim that the boundary rate $R = R_I(P,d,D)$ is achievable. Such a statement requires an additional closure or compactness argument for the operational achievable region, or a more specialized endpoint theorem. Keeping strict achievability as the main direct statement matches the proof actually formalized and prevents an unjustified per-block rate claim at the infimum.

12 Reusable infrastructure

The following components are useful beyond the final theorem.

Infrastructure	Reusable content
Finite PMF calculus	Real expectation and variance, map and bind identities, normalization, product formulas, finite Chebyshev inequality.
Entropy API	Entropy under maps, entropy cardinality bounds, conditional entropy, strong subadditivity, coordinate conditional-entropy bounds.
KL/entropy bridge	Explicit finite density ratios, absolute continuity, Radon–Nikodym identification, and equality of KL and entropy mutual information.
Channel reconstruction	Conditional PMFs and reconstruction of arbitrary joint PMFs as induced source/channel joint laws.
Channel mixtures	PMF and channel mixtures, linearity of distortion, convexity of mutual information, and convexity of R_I.
Memoryless blocks	I.i.d. block PMFs, memoryless channels, paired-block equivalence, coordinate marginals, entropy and mutual-information additivity.
Block statistics	Expectation and variance of block sums and averages, with weak-law estimates for arbitrary real observables.
Random covering	Codebook hit/miss probabilities, exact i.i.d. miss powers, likelihood-ratio change of measure, exponential miss bounds.

Infrastructure	Reusable content
Probabilistic method	Extraction of a deterministic codebook from an average random-codebook estimate.
Code single-letterization	Uniform-coordinate mixtures and a finite-block rate–distortion converse for arbitrary deterministic codes.
Asymptotic filters	Limsup rate and distortion, eventual/frequent bounds, right-continuity transfer.
Diagonalization	A generic construction converting levelwise eventual objects with vanishing errors into one asymptotically exact sequence.

Several design patterns are especially suitable for a broader Lean information-theory library:

1. use `PMF.map` and `PMF.bind` as the primary finite-law combinators;
2. keep real entropy algebra separate from extended-nonnegative KL values, connected by an explicit bridge theorem;
3. package arbitrary joint laws through marginals and conditional PMFs;
4. state concentration lemmas for generic real observables before specializing them to information density and distortion;
5. expose finite-block quantitative coding theorems separately from their asymptotic corollaries;
6. isolate diagonal selection from coding-specific mathematics.

13 Mathlib reuse and type discipline

The development reuses the following mathlib abstractions directly:

Mathematical role	Mathlib representation
finite probability law	`PMF`
deterministic pushforward	`PMF.map`
conditional composition	`PMF.bind`
uniform time-sharing variable	`PMF.uniformOfFintype`
finite blocks and coordinates	`Fin n`, function types
finite sums and products	`Finset`, `Fintype`
nonnegative finite distortion	`NNReal`
expectations, thresholds, infima	`ENNReal`
KL divergence and log likelihood	`InformationTheory.klDiv`, `llr`
concavity of entropy integrand	`Real.concaveOn_negMulLog`
asymptotic order	`Filter.limsup`, `atTop`
right limits	`Tendsto`, `nhdsWithin`
infimum definition	complete-lattice `sInf`

No custom probability-measure structure, finite-index structure, extended number system, or asymptotic filter is introduced. Conversely, the project does define a finite-sum expectation interface because it is lighter and more effective for the combinatorial concentration proof than repeatedly installing finite measurable-space instances.

The use of explicit predicates is also deliberate. Feasibility, interior distortion, positive blocklength, nonempty codebooks, and finite distortion thresholds appear as hypotheses or witnesses. They are not hidden in typeclass inference, because they describe the domain of a particular theorem rather than canonical structure on an alphabet.

14 Verification and reproducibility

The source module was rebuilt on 14 August 2026 with:

```
lake build Myproject.RateDistortion.RateDistortion
```

The build completed successfully with 3156 jobs. The file contains no `sorry`. A refreshed `#print axioms` audit gave the same result for all three public endpoints:

```
rate_distortion_achievability:
  [propext, Classical.choice, Quot.sound]

rate_distortion_converse:
  [propext, Classical.choice, Quot.sound]

rate_distortion_nonachievability:
  [propext, Classical.choice, Quot.sound]
```

In particular, none depends on `sorryAx`. The remaining build messages are linter warnings, principally unused section variables inherited from the file-wide finite-alphabet context; they do not indicate proof gaps.

The current source has approximately 5690 lines, 297 top-level declarations, and 224 named theorems or lemmas. These counts describe the present monolithic file and should not be treated as stable API metrics after modularization.

15 Current scope and extension path

Layer	Status
finite-alphabet PMF expectation and variance	implemented and checked
Shannon and conditional entropy	implemented and checked
strong subadditivity and conditioning inequalities	implemented and checked
KL/entropy mutual-information bridge	implemented and checked
single-letter $R_I(P, d, D)$ and feasibility notions	implemented and checked
monotonicity, convexity, finiteness, interior right continuity	implemented and checked
i.i.d. block sources and memoryless test channels	implemented and checked
information-density weak law	implemented and checked
random codebook covering and deterministic extraction	implemented and checked
quantitative finite-block direct theorem	implemented and checked
arbitrary-code single-letter converse	implemented and checked
code sequences, limsup, and diagonalization	implemented and checked
strict achievability $R > R_I(P, d, D)$	implemented and checked
interior converse and strict nonachievability	implemented and checked
boundary achievability $R = R_I(P, d, D)$	not claimed; future endpoint analysis

Layer	Status
converse at every feasible endpoint	only the $D + \varepsilon$ form is unconditional; exact D currently uses the interior right-continuity theorem
countable or continuous alphabets	outside the present finite-PMF scope
unbounded or infinite letterwise distortion	outside the present `NNReal`-valued distortion model
stochastic operational encoders/decoders	outside the present deterministic-code API
public module decomposition and downstream API tests	recommended next infrastructure step

The most useful next engineering step is modularization without changing the mathematical API. A natural file structure is:

```
RateDistortion/
  FinitePMF.lean
  Entropy.lean
  MutualInformation.lean
  Function.lean
  Memoryless.lean
  RandomCoding.lean
  Converse.lean
  Asymptotics.lean
  Theorem.lean
```

Focused public-import tests should then ensure that clients can use the main definitions and theorem endpoints without importing implementation modules.

On the mathematical side, the equality-rate question should be treated as a separate theorem rather than folded into the current strict direct statement. Possible approaches include proving closure of the operational achievable region under the chosen limsup definition, establishing attainment and a carefully controlled sequence of rates, or proving endpoint results under additional hypotheses. The present infrastructure already supplies the finite-block codes and the diagonal machinery needed for such an investigation, but it does not by itself turn $R_k \downarrow R_I(D)$ into codes whose every block rate is bounded by exactly $R_I(D)$.

16 Conclusion

The rate–distortion formalization is now a complete checked reconstruction of the strict finite-alphabet theorem within its stated domain. Its main value is not only the final declarations. The proof has produced reusable APIs for finite entropy, KL conversion, channel mixtures, i.i.d. blocks, information density, concentration, random covering, deterministic codebook extraction, time-sharing single-letterization, limsup reasoning, and diagonal selection.

The representation choices keep the mathematics honest: finite letterwise distortion is separated from extended thresholds, zero blocklength is excluded by construction, infima are not assumed attained, and right continuity is proved exactly where it is used. The final theorem states strict achievability, while equality at $R = R_I(P, d, D)$ remains visible as a distinct endpoint problem.

The source, dependency chain, quantitative intermediate bounds, successful Lake build, and axiom audit make the result reviewable at three levels: mathematical meaning, Lean API design, and machine-checked proof.

References

[1] Leonardo de Moura and Sebastian Ullrich. *The Lean 4 Theorem Prover and Programming Language*.

[2] The mathlib Community. *The mathlib Mathematical Library*.

[3] Claude E. Shannon. *Coding Theorems for a Discrete Source with a Fidelity Criterion*.

[4] Thomas M. Cover and Joy A. Thomas. *Elements of Information Theory*, second edition.